Question 2 Problem Statement: Enhanced Forward/Backward Reasoning — Deceptive Rules and Sparse Inference

Q2.1: Logic Translation with Mixed Relevance

Convert the following 10 statements into propositional logic, using the symbol key:

- **S:** is Sick
- **V:** likely to Vomit
- **P:** looks Pale
- **T:** is Thirsty
- **H:** has High Temperature
- **NS:** has Not Slept well
- **NF:** does Not Feel well
- **F:** has Fever
- **W:** drinks Water
- **E:** is Exhausted
- **A:** has Allergy
- **M:** has Muscle Pain

Note: If the knowledge base does not have the symbol key, it means that you can ignore it.

- **R1.** If a person looks pale, is thirsty, and has not slept well, then they are sick.
- **R2.** If a person has a fever, then they have a high temperature.
- **R3.** If a person has a high temperature, then they are thirsty.
- **R4.** If a person is exhausted, then they have not slept well.
- **R5.** If a person has not slept well, then they look pale.
- **R6.** If a person looks pale, then they should drink water.
- **R7.** If a person has muscle pain, then they are not feeling well.
- **R8.** A person has an allergy if they sneeze in spring.
- **R9.** If a person is not thirsty and has slept well, then they either will not vomit or will drink water.
- **R10.** If a person is not exhausted and not thirsty, then they might either feel well or drink water.

Initial facts (observations):

- The person has a fever.

- The person is exhausted.

Task: Prove that the person is sick (S)

Q2.2: Forward Chaining
Refer to our lab format to complete the forward chaining proving process.

Q2.3: Backward Chaining
Refer to our lab format to complete the backward chaining proving process.

Q2.4 Discussion
(i) Does forward chaining stop early in this scenario? Why?
(ii) Could adding $\neg M$ or A improve inference? Why or why not?

Q2.1: Logic Translation with Mixed Relevance

> **Lecture Recap:**
>
> - **Propositional Logic:** We translate English sentences into formal logic using logical connectives:
> - Conjunction: AND / $\wedge$
> - Disjunction: OR / $\vee$
> - Implication: IF-THEN / $\Rightarrow$
> - Negation: NOT / $\neg$
> - **Implication:** A sentence structured as "**If A, then B**" or "B **if** A" is translated as $A \Rightarrow B$.
> - **Handling Unrepresented Symbols:** According to the assignment instructions, if a statement contains elements not present in our designated symbol key, we must ignore it. This prevents the Knowledge Base (KB) from containing undefined or open variables that cannot be evaluated.

Propositional Logic Translation:

- **R1:** If a person looks pale (P), is thirsty (T), and has not slept well (NS), then they are sick (S).
  Formula: $P \wedge T \wedge NS \Rightarrow S$
- **R2:** If a person has a fever (F), then they have a high temperature (H).
  Formula: $F \Rightarrow H$

- **R3:** If a person has a high temperature (H), then they are thirsty (T).
  Formula: $H \Rightarrow T$
- **R4:** If a person is exhausted (E), then they have not slept well (NS).
  Formula: $E \Rightarrow NS$
- **R5:** If a person has not slept well (NS), then they look pale (P).
  Formula: $NS \Rightarrow P$
- **R6:** If a person looks pale (P), then they should drink water (W).
  Formula: $P \Rightarrow W$
- **R7:** If a person has muscle pain (M), then they are not feeling well (NF).
  Formula: $M \Rightarrow NF$
- **R8:** A person has an allergy (A) if they sneeze in spring.
  Formula: IGNORED.
  *Observation: "**Sneeze in spring**" is not in the provided symbol key. Per the assignment rules, because we lack a symbol for the premise, this rule must be discarded from our formal KB.*
- **R9:** If a person is not thirsty ($\neg T$) and has slept well (negation of NS: $\neg NS$), then they either will not vomit ($\neg V$) or will drink water (W).
  Formula: $\neg T \wedge \neg NS \Rightarrow \neg V \vee W$
- **R10:** If a person is not exhausted ($\neg E$) and not thirsty ($\neg T$), then they might either feel well (negation of NF: $\neg NF$) or drink water (W).
  Formula: $\neg E \wedge \neg T \Rightarrow \neg NF \vee W$

Initial Facts (Observations):

- The person has as fever (F)
- The person is exhausted (E)

Goal:

- Prove that the person is sick (S)

Q2.2: Forward Chaining

> **Lecture Recap:**
>
> - **Forward Chaining** is a **data-driven** algorithm. It starts with the known facts and applies **Modus Ponens** iteratively to trigger rules and generate new facts until the goal is reached or no more facts can be inferred.

- **Definite Clauses:** Forward chaining expects the knowledge base to consist of Definite Clauses (rules with exactly one positive literal in the conclusion). Rules $R9$ and $R10$ contain disjunctions (OR / $\vee$) in their conclusions, meaning they are not Horn clauses and cannot be processed by standard forward chaining. We will evaluate the counts for the valid definite clauses ($R1$ through $R7$).
- **Algorithm Mechanics:** We maintain a `Count` of how many premises are unmet for each rule. When a fact is popped from the `Queue`, we decrement the count for any rule requiring that fact. If a rule's count reaches 0, its conclusion is added to the `Queue`.

Execution Steps:

- **Initial state:**
  We initialize the `Count` for the premises of our definite clauses:
  - $R1$ has 3 premises (P, T, NS).
  - $R2, R3, R4, R5, R6, R7$ each has 1 premise.
- **Queue:**
  Populated with the initial facts: $[F, E]$.

Step	Queue	Count	Inferred	Action / Rule Triggered
0	F, E	R1:3, R2:1, R3:1, R4:1, R5:1, R6:1, R7:1		*Initial State*
1	E	R1:3, R2:0, R3:1, R4:1, R5:1, R6:1, R7:1	F	Pop F. R2 count reaches 0. Infer H. Add H to Queue.
2	E, H	R1:3, R2:0, R3:1, R4:0, R5:1, R6:1, R7:1	F, E	Pop E. R4 count reaches 0. Infer NS. Add NS to Queue.
3	H, NS	R1:3, R2:0, R3:0, R4:0, R5:1, R6:1, R7:1	F, E, H	Pop H. R3 count reaches 0. Infer T. Add T to Queue.

Step	Queue	Count	Inferred	Action / Rule Triggered
4	NS, T	R1:2, R2:0, R3:0, R4:0, R5:0, R6:1, R7:1	F, E, H, NS	Pop NS. R1 count drops to 2. R5 count reaches 0. Infer P. Add P to Queue.
5	T, P	R1:1, R2:0, R3:0, R4:0, R5:0, R6:1, R7:1	F, E, H, NS, T	Pop T. R1 count drops to 1.
6	P	R1:0, R2:0, R3:0, R4:0, R5:0, R6:0, R7:1	F, E, H, NS, T, P	Pop P. R1 count reaches 0. Infer S. R6 count reaches 0. Infer W. Add S, W to Queue.
7	S, W	R1:0, R2:0, R3:0, R4:0, R5:0, R6:0, R7:1	F, E, H, NS, T, P, S	Pop S. **Goal Reached!**

🎯 **Conclusion:** The algorithm successfully proves S (the person is Sick).

Q2.3: Backward Chaining

Lecture Recap:

- **Backward Chaining** is a **goal-driven** algorithm. It works backward from the query q (the Goal).
- **Mechanism:** To prove a goal, the algorithm checks if it is already a known fact. If not, it finds all rules that have the goal as their conclusion (the right-hand side of the $\Rightarrow$) and recursively attempts to prove every premise of those rules (the left-hand side) as new subgoals.

- **Efficiency:** It avoids loops by checking if a subgoal is already on the stack, and it avoids repeated work by checking if a subgoal has already been proven true in a previous branch.

Execution Steps:

1. **Current Goal:** S
   - Is S a known fact? No.
   - Find a rule concluding S: **R1** ($P \wedge T \wedge NS \Rightarrow S$).
   - To prove S, we must prove the subgoals: P, T, NS.
2. **Evaluate Subgoal 1:** P
   - Is P a known fact? No.
   - Find a rule concluding P: **R5** ($NS \Rightarrow P$).
   - New Subgoal to prove P: NS.
   - **Evaluate Subgoal 1.1:** NS
     - Is NS a known fact? No.
     - Find a rule concluding NS: **R4** ($E \Rightarrow NS$).
     - New Subgoal to prove NS: E.
     - **Evaluate Subgoal 1.1.1:** E
       - Is E a known fact? **Yes!**
     - *Result:* E is true $\rightarrow$ NS is true $\rightarrow$ P **is true**.
3. **Evaluate Subgoal 2:** T (Returning to R1's premises)
   - Is T a known fact? No.
   - Find a rule concluding T: **R3** ($H \Rightarrow T$).
   - New Subgoal to prove T: H.
   - **Evaluate Subgoal 2.1:** H
     - Is H a known fact? No.
     - Find a rule concluding H: **R2** ($F \Rightarrow H$).
     - New Subgoal to prove H: F.
     - **Evaluate Subgoal 2.1.1:** F
       - Is F a known fact? **Yes!**
     - *Result:* F is true $\rightarrow$ H is true $\rightarrow$ T **is true**.
4. **Evaluate Subgoal 3:** NS (Returning to R1's premises)
   - Is NS a known fact? No.
   - *Theory Check:* Has this subgoal already been proven? **Yes.** We proved NS in Step 2 while proving P.
   - *Result:* The algorithm immediately returns true without recalculating. NS **is true**.

🎯 **Conclusion:** All three premises (P, T, NS) of R1 evaluate to true. Therefore, the goal S is proven to be true.

Q2.4: Discussion

(i) Does forward chaining stop early in this scenario? Why?

Yes, forward chaining stops early.

- **Justification:**
  The forward chaining algorithm includes an explicit termination condition:
  - `if p == q then return true`
    - p is the fact currently popped from the queue
    - q is the goal.
  - In Step 6 of our trace, processing P inferred both S and W. Both were added to the queue.
  - In Step 7, S was popped from the queue. Because S matched our goal, the algorithm successfully stopped immediately.
  - It *stopped early* because it did not waste computational resources evaluating the remaining items in the queue (like popping W) or attempting to trigger further rules down the line.
  - It also naturally ignored irrelevant paths (like $R7$) because they were never triggered by the data.

(ii) Could adding $\neg M$ or A improve inference? Why or why not?

No, adding $\neg M$ or A would not improve inference.

- **Justification:**
  - **Relevance to Goal:** The goal S is already perfectly provable using only the minimal initial facts (F, E). Adding more facts to the KB cannot make the proof of S any more successful; it would only expand the state space and the queue size, mathematically *decreasing* algorithmic efficiency.
  - **Structural Limitations:** Standard Forward and Backward Chaining algorithms are designed to operate exclusively on Horn Knowledge Bases (Definite Clauses containing positive literals). Adding a negative literal ($\neg M$) cannot be utilized by basic chaining rules, and rule $R7$ requires a positive M to trigger.

- **Disconnected Rules:** The symbol A relies on rule $R8$. As established in Q2.1, R8 is discarded because its premise ("**sneeze in spring**") is unrepresented. Consequently, A is entirely disconnected from the logic network mapping to S. Injecting it provides zero heuristic or logical value.